Design an Object-Oriented Python Application



KEY IDEAS

1. Requirements in plain language: what must the system do?

2. Nouns → classes, verbs → methods, adjectives → attributes.

3. One responsibility per class; decide relationships: is-a (inherit), has-a (compose), uses (depend).

4. Sketch with CRC cards or a quick UML diagram.

5. Build the core model first, then services, then I/O; test as you go (`pytest`).

6. Refactor: kill duplication, depend on abstractions.

	SOLID	In practice
S	Single responsibility	one reason to change
O	Open / closed	extend by adding classes
L	Liskov substitution	subclass works wherever base does
I	Interface segregation	small ABCs / Protocols
D	Dependency inversion	depend on abstractions; inject parts

```
library_app/
  models.py     # Book, Member      (data)
  services.py   # Library           (logic)
  storage.py    # save / load       (I/O)
  main.py       # entry point
  tests/        # pytest
```

TIP Compose small collaborators and **inject** them (`Library(storage=JsonStorage())`) so you can swap or mock them in tests.

EXAMPLE

```python
from dataclasses import dataclass

@dataclass
class Book:                             # data holder
    isbn: str
    title: str
    available: bool = True

class Member:                           # borrower
    def __init__(self, name):
        self.name = name
        self.loans = []                 # has-many Book

class Library:                          # orchestrator
    def __init__(self):
        self._books = {}                # isbn → Book

    def add(self, book):
        self._books[book.isbn] = book

    def checkout(self, isbn, member):
        book = self._books[isbn]
        if not book.available:
            raise ValueError(f"{book.title} is out")
        book.available = False
        member.loans.append(book)
        return book
```

```python
lib, ann = Library(), Member("Ann")
lib.add(Book("111", "Fluent Python"))
lib.checkout("111", ann)
print([b.title for b in ann.loans])  # → ['Fluent Python']
try:
    lib.checkout("111", Member("Bo"))
except ValueError as e:
    print(e)                         # → Fluent Python is out
```